

COMPLAINT MANAGEMENT SYSTEM

Project Documentation

A Java Full Stack Web Application

React.js | Spring Boot | MySQL

Objective of the Project

To design and develop a secure, role-based complaint and grievance redressal system that allows students to register, authenticate, and submit complaints, while enabling administrators to track, assign, and resolve them through a modern web interface backed by a robust REST API and relational database.

1. Introduction

1.1 What is a Complaint Management System?

A complaint management system is a web application that allows an organization — such as a college — to receive, categorize, track, and resolve complaints or grievances raised by its members. It typically provides features such as user registration, secure login, structured complaint submission, and real-time status tracking, replacing informal or paper-based reporting with an auditable digital process.

1.2 Why This Project is Useful

This project demonstrates how modern web technologies can be combined to build a complete, production-style application. It gives learners hands-on experience with frontend development, backend REST API design, database management, and role-based authentication — all essential skills for a Full Stack Developer. It also serves as a strong portfolio project that can be extended with advanced features such as notifications or analytics.

1.3 Users of the System

- Admin — Manages staff allocation, oversees every submitted complaint across the college, and drives cases to resolution.
- Student (Registered User) — Can create an account, log in securely, submit complaints with category and priority, and track their status until resolution.

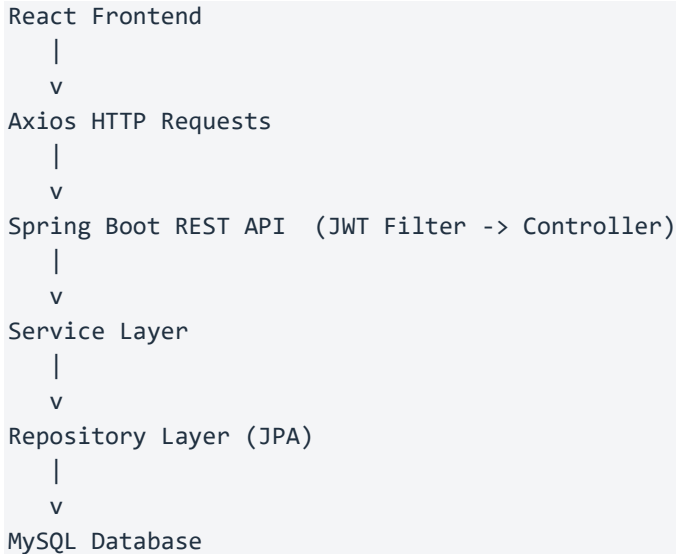
2. Technologies Used

Technology	Purpose
React.js	Building the dynamic frontend user interface
Tailwind CSS	Structuring and styling web pages
JavaScript	Client-side logic and interactivity
Axios	Sending HTTP requests from React to the backend API
React Router	Handling navigation between frontend pages
Java	Core programming language for the backend
Spring Boot 3	Building the REST API and backend application
Spring Data JPA	Simplifying database operations using ORM (Hibernate)
Spring Security + JWT	Securing endpoints and handling role-based authentication
Maven	Managing backend project dependencies and build
MySQL	Storing application data (users and complaints)
IntelliJ IDEA	Backend development environment
Visual Studio Code	Frontend development environment
MySQL Workbench	Designing and managing the database

3. Project Architecture

The application follows a layered client-server architecture. The React frontend runs in the user's browser and communicates with the Spring Boot backend through REST APIs over HTTP. The backend processes requests through its own internal layers, with a JWT filter enforcing role-based access, before reading from or writing to the MySQL database.

3.1 Data Flow Diagram



3.2 Layer Responsibilities

- React Frontend: Renders the UI and captures user actions such as complaint form submissions.
- Axios: Sends HTTP requests (GET, POST, PUT, DELETE) from React to the backend and returns responses.
- REST API (Controller): Receives HTTP requests and routes them to the correct service method, protected by role-based access rules.
- Service Layer: Contains the business logic — validation, categorization, status transitions, and coordination.
- Repository Layer (JPA): Converts Java objects into SQL queries and communicates with the database.
- MySQL Database: Stores and retrieves persistent data such as users and complaints.

4. Project Development Process

The project is built in the following sequential steps:

1. Create a Spring Boot project using Spring Initializr.
2. Add dependencies: Spring Web, Spring Data JPA, MySQL Driver, Spring Security, JWT.
3. Configure database connection details in application.properties.
4. Create the database schema in MySQL.
5. Create entity classes: User, Complaint.
6. Create repository interfaces for database access.
7. Create service classes containing business logic and status-handling rules.
8. Create REST controllers to expose API endpoints, secured with a custom JWT filter.
9. Test backend APIs using Postman.
10. Create the React project using Vite.
11. Install required packages: axios and react-router-dom.
12. Create components: Login, Register, Student Dashboard, Complaint Form, Admin Panel, Navbar.
13. Connect React with Spring Boot using Axios and a request interceptor for the JWT.
14. Run the backend application.
15. Run the frontend application.

5. Backend and Frontend Connection

React and Spring Boot communicate through REST APIs using JSON as the data exchange format. When a user interacts with the browser, the request travels through several layers before a response is returned.



5.1 HTTP Methods Used

Method	Purpose
GET	Retrieve data, such as fetching the list of complaints
POST	Create new data, such as registering a user or filing a complaint
PUT	Update existing data, such as assigning staff or changing status
DELETE	Remove data, such as deleting a withdrawn complaint

5.2 JSON Request and Response

Data is exchanged between React and Spring Boot in JSON format. When the frontend sends a request — such as filing a complaint — it sends a JSON object containing fields like category, priority, and description. The backend processes this data and returns a JSON response, either the created resource or a status message, which React then uses to update the dashboard.

6. Database Design

6.1 Users Table

Column	Description
id	Unique identifier for each user (Primary Key)
name	Full name of the student or admin
email	Unique email address used as the login identifier
password	Encrypted (BCrypt) password used for authentication
role	ROLE_STUDENT or ROLE_ADMIN

6.2 Complaints Table

Column	Description
id	Unique identifier for each complaint (Primary Key)
title / description	Details of the complaint entered by the student
category	Hostel, Academics, Library, Transport, or Others
priority	Low, Medium, or High
status	Pending, Assigned, or Resolved
assigned_staff	Staff member allocated by the admin
created_date	Date and time the complaint was submitted
user_id	References the student who filed the complaint (Foreign Key)

A user can file many complaints, forming a one-to-many relationship between users and complaints, linked via the user_id foreign key.

7. Application Workflow

The end-to-end working of the application follows this sequence:

```
Student Registration -> Login using JWT -> Dashboard -> Submit Complaint  
-> Store Complaint in MySQL -> Admin Views & Assigns Staff  
-> Status Updates (Pending -> Assigned -> Resolved) -> Student Tracks Status -> Logout
```

8. Features

Category	Capabilities
Authentication	User Registration, Login Authentication, JWT-based Security
Complaint Management	Submit, Categorize, Prioritize, and Track Complaints
Admin Tools	Master List View, Multi-Category Filters, Staff Allocation
Platform	Responsive User Interface, RESTful APIs, Persistent MySQL Storage

9. Advantages

- Provides hands-on experience with the complete full-stack development lifecycle.
- Uses industry-standard technologies (React, Spring Boot, MySQL) valued by employers.
- Secure authentication using JWT protects user data and API endpoints.
- Layered architecture improves code organization, readability, and maintainability.
- REST API design allows the frontend and backend to evolve independently.
- Scalable foundation that can be extended with new features or deployed to the cloud.

10. Future Enhancements

- File / image attachments as evidence for complaints.
- Email or SMS notifications on status changes.
- Analytics dashboard for complaint trends and resolution times.
- In-app comments or chat between student and assigned staff.
- Multi-level escalation for unresolved high-priority complaints.
- Mobile application for on-the-go complaint tracking.

11. Conclusion

The Complaint Management System project demonstrates how React, Spring Boot, and MySQL work together to build a secure, scalable, and maintainable full-stack web application. React delivers a fast and interactive user interface, Spring Boot exposes secure REST APIs protected by JWT authentication and role-based access control, and MySQL provides reliable, persistent data storage. Together, these technologies form a solid foundation for real-world grievance-handling systems and serve as an excellent, extensible portfolio project for aspiring full-stack developers.